STA 325 Case Study: Particle Clusters in Turbulence

Johan Nino Espino

2026-10-25

Setup

Goal. Predict four summaries of the particle cluster-volume distribution (mean, variance, skewness, kurtosis) from the Reynolds number Re, the Froude number Fr and the Stokes number St, and explain how each input affects them. The emphasis is on interpretable models that still predict well.

Put `data-train.csv` and `data-test.csv` in a folder called `data` next to this file.

```
library(ggplot2)
library(dplyr)
```

```
Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

    filter, lag

The following objects are masked from 'package:base':

    intersect, setdiff, setequal, union
```

```
library(tidyr)
library(glmnet)
```

```
Warning: package 'glmnet' was built under R version 4.5.2
```

```
Loading required package: Matrix
```

```
Attaching package: 'Matrix'
```

```
The following objects are masked from 'package:tidyr':

    expand, pack, unpack
```

```
Loaded glmnet 5.1
```

```
theme_set(theme_bw(base_size = 10))
```

Load the data

```
train <- read.csv("data/data-train.csv")
test  <- read.csv("data/data-test.csv")
dim(train); dim(test)
```

```
[1] 89  7
```

```
[1] 23  3
```

```
colSums(is.na(train))   # no missing values; read.csv reads "Inf" as Inf
```

```
        St         Re         Fr R_moment_1 R_moment_2 R_moment_3 R_moment_4
         0          0          0          0          0          0          0
```

Convert raw moments to the four targets

The data give the raw moments $m_k = E[X^k]$. With $\mu = m_1$, expanding $E[(X-\mu)^k]$ gives the central moments:

$$\sigma^2 = m_2 - m_1^2, \qquad \mu_3 = m_3 - 3m_1m_2 + 2m_1^3, \qquad \mu_4 = m_4 - 4m_1m_3 + 6m_1^2m_2 - 3m_1^4,$$

so skewness $= \mu_3/\sigma^3$ and kurtosis $= \mu_4/\sigma^4$.

```r
m1 <- train$R_moment_1; m2 <- train$R_moment_2
m3 <- train$R_moment_3; m4 <- train$R_moment_4

train$mean     <- m1
train$variance <- m2 - m1^2
mu3 <- m3 - 3 * m1 * m2 + 2 * m1^3
mu4 <- m4 - 4 * m1 * m3 + 6 * m1^2 * m2 - 3 * m1^4
train$skewness <- mu3 / train$variance^1.5
train$kurtosis <- mu4 / train$variance^2

# Checks: variance must be positive; all four targets positive (for logs)
sum(train$variance <= 0)
```

```
[1] 0
```

```r
sapply(train[, c("mean", "variance", "skewness", "kurtosis")],
       function(x) sum(x <= 0))
```

```
    mean variance skewness kurtosis
       0        0        0        0
```

Encode the inputs

Re and Fr each take only three values, so both are treated as categories (factors). St takes 14 values and stays numeric.

```r
encode <- function(d) {
  d$Re_f <- factor(d$Re, levels = c(90, 224, 398))
  d$Fr_f <- factor(d$Fr, levels = c(0.052, 0.3, Inf),
                   labels = c("0.052", "0.3", "Inf"))
  d$log_St <- log(d$St)
  d
}
train <- encode(train)
test  <- encode(test)

table(Re = train$Re_f, Fr = train$Fr_f)   # training runs per setting
```

```
     Fr
Re    0.052 0.3 Inf
```

```
  90      12  10   9
  224     14  13  12
  398     10   0   9
```

```
table(Re = test$Re_f,  Fr = test$Fr_f)    # test runs per setting
```

```
     Fr
Re    0.052 0.3 Inf
  90      2   4   5
  224     0   1   2
  398     4   0   5
```

Every test setting of (Re, Fr) also appears in training. There are no runs at Re = 398 with Fr = 0.3.

Exploratory data analysis

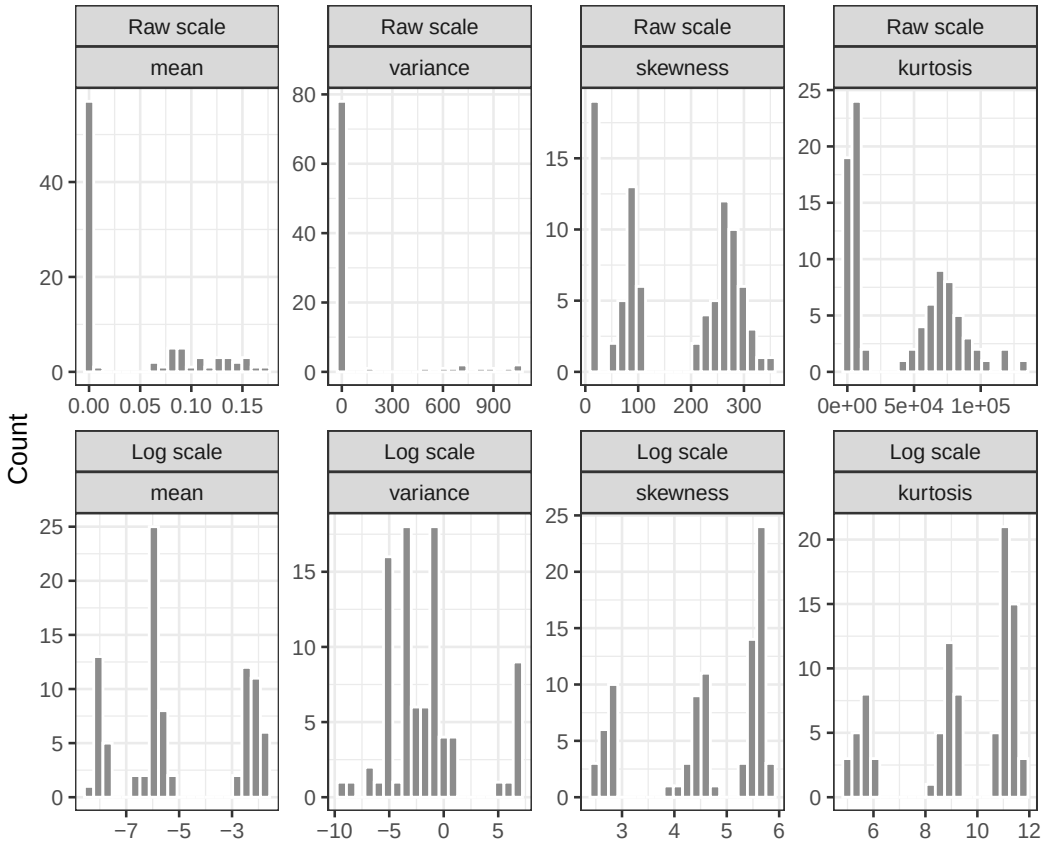
```
targets <- c("mean", "variance", "skewness", "kurtosis")
fr_cols <- c("0.052" = "#D55E00", "0.3" = "#0072B2", "Inf" = "#009E73")

long <- train |>
  select(St, log_St, Re_f, Fr_f, all_of(targets)) |>
  pivot_longer(all_of(targets), names_to = "target", values_to = "value") |>
  mutate(target = factor(target, levels = targets))
```

Each target on its own: is a transformation needed?

```
hist_df <- bind_rows(
  mutate(long, scale = "Raw scale", x = value),
  mutate(long, scale = "Log scale", x = log(value))
) |> mutate(scale = factor(scale, levels = c("Raw scale", "Log scale")))

ggplot(hist_df, aes(x)) +
  geom_histogram(bins = 20, fill = "grey55", color = "white") +
  facet_wrap(scale ~ target, scales = "free", nrow = 2) +
  labs(x = NULL, y = "Count")
```




```
# How many times larger is the largest value than the smallest?
sapply(train[, targets], function(x) signif(max(x) / min(x), 2))
```

```
    mean variance skewness kurtosis
 7.8e+02  1.0e+07  2.9e+01  8.8e+02
```

```
# Spread within each (Re, Fr) setting, raw vs log scale. A log transform is
# justified if it makes the spread similar across settings.
long |>
  group_by(target, Re_f, Fr_f) |>
  summarise(raw_sd = sd(value), log_sd = sd(log(value)), .groups = "drop") |>
  group_by(target) |>
  summarise(raw_sd_ratio = signif(max(raw_sd) / min(raw_sd), 2),
            log_sd_ratio = signif(max(log_sd) / min(log_sd), 2))
```

```
# A tibble: 4 x 3
```

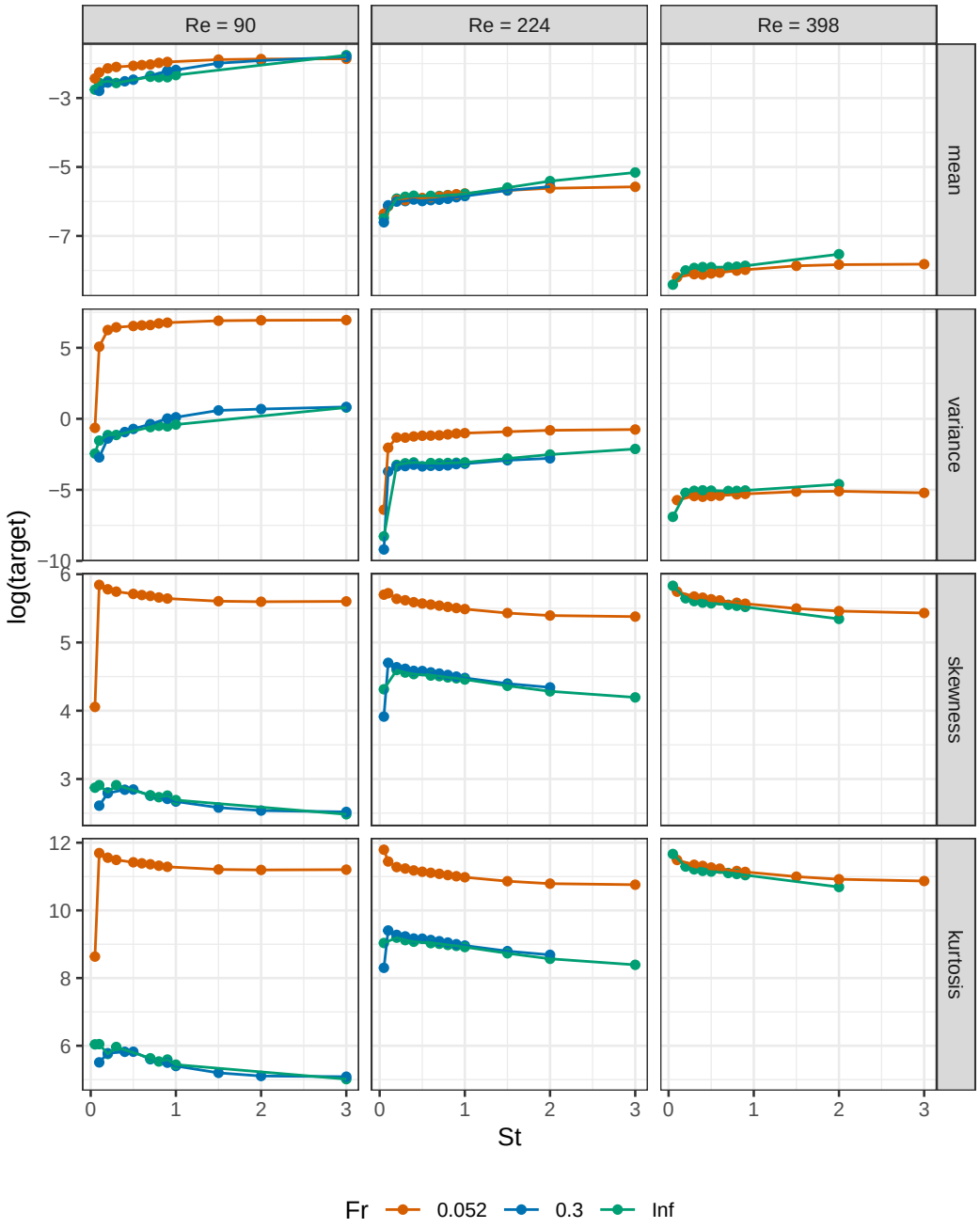
```
  target   raw_sd_ratio log_sd_ratio
  <fct>           <dbl>        <dbl>
1 mean              750          2.4
2 variance       380000         11
3 skewness           40          4.8
4 kurtosis          400          4.1
```

Decision: model the log of every target. On the raw scale the targets span up to about ten million-fold and their spread differs enormously between settings; on the log scale the spread is comparable. The log scale also turns the multiplicative effects (for example, Re changes the mean roughly ten-fold per step) into additive ones, and back-transformed predictions are always positive.

```
train <- train |>
  mutate(log_mean = log(mean), log_variance = log(variance),
         log_skewness = log(skewness), log_kurtosis = log(kurtosis))
responses <- c(Mean = "log_mean", Variance = "log_variance",
               Skewness = "log_skewness", Kurtosis = "log_kurtosis")
```

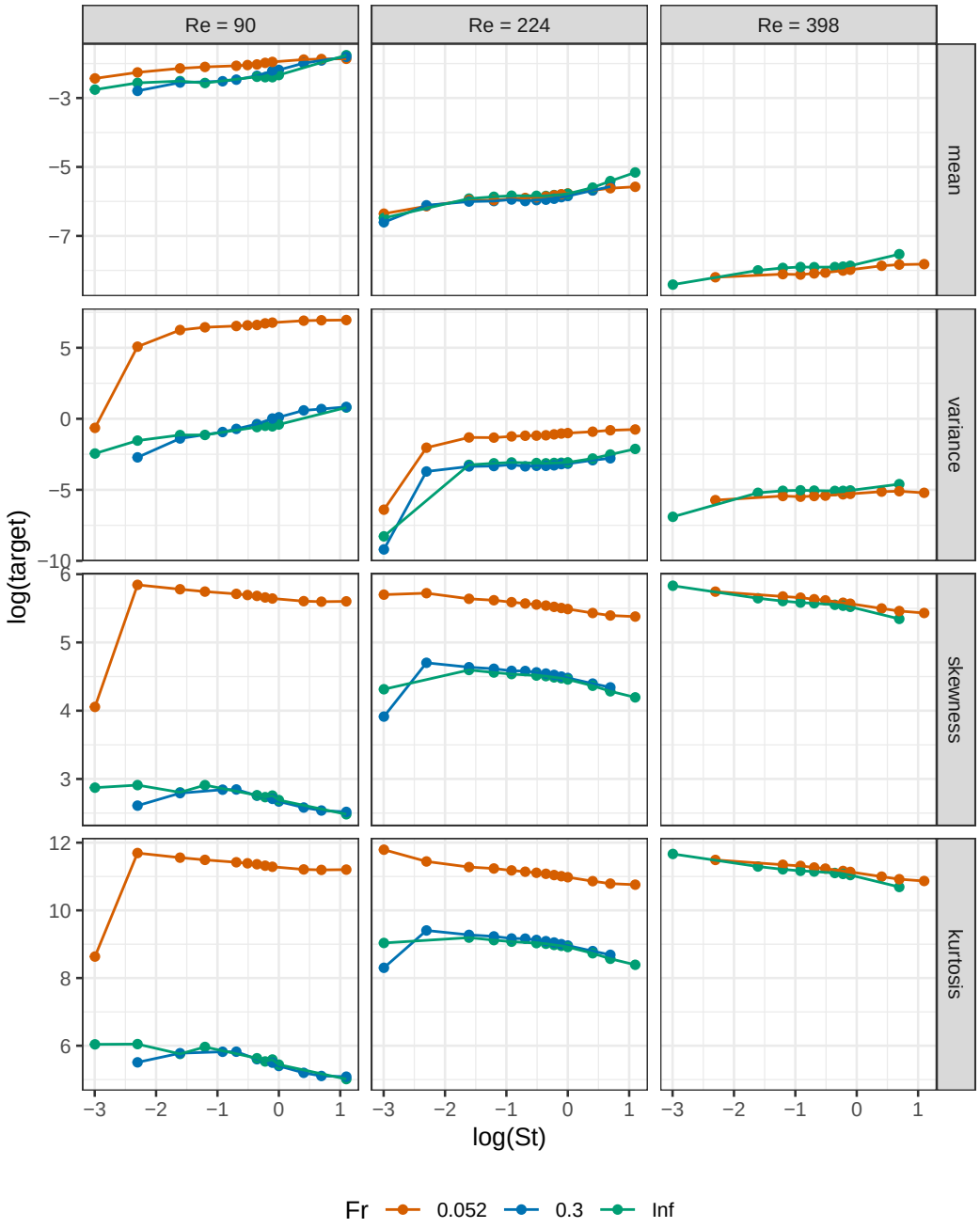
Nonlinearity: how does each target change with St?

```
ggplot(long, aes(St, log(value), color = Fr_f)) +
  geom_point(size = 1.3) + geom_line() +
  facet_grid(target ~ Re_f, scales = "free_y",
             labeller = labeller(Re_f = function(x) paste("Re =", x))) +
  scale_color_manual(values = fr_cols, name = "Fr") +
  labs(x = "St", y = "log(target)") +
  theme(legend.position = "bottom")
```




```
ggplot(long, aes(log_St, log(value), color = Fr_f)) +
  geom_point(size = 1.3) + geom_line() +
  facet_grid(target ~ Re_f, scales = "free_y",
             labeller = labeller(Re_f = function(x) paste("Re =", x))) +
```

```
  scale_color_manual(values = fr_cols, name = "Fr") +
  labs(x = "log(St)", y = "log(target)") +
  theme(legend.position = "bottom")
```

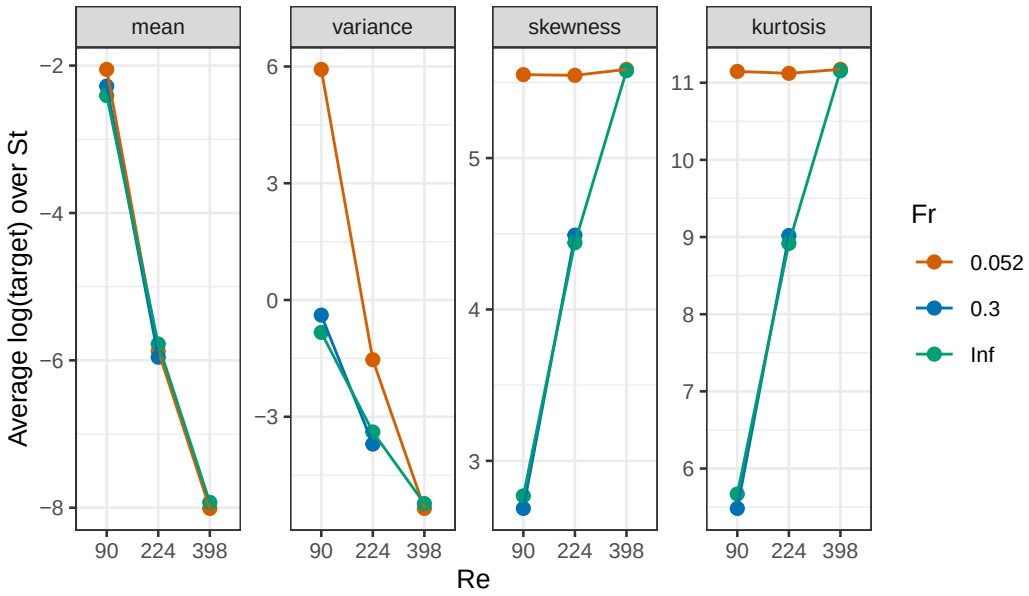


Every curve changes sharply between St = 0.05 and 0.2 and then slowly, so the St effect is clearly **nonlinear**. Using log(St) spreads out the small St values and makes the curves smoother, but they still bend. This suggests log(St) plus polynomial terms.

Interactions: does the effect of one input depend on another?

```
cell_means <- long |>
  group_by(target, Re_f, Fr_f) |>
  summarise(avg = mean(log(value)), .groups = "drop")

ggplot(cell_means, aes(Re_f, avg, color = Fr_f, group = Fr_f)) +
  geom_point(size = 2) + geom_line() +
  facet_wrap(~ target, scales = "free_y", nrow = 1) +
  scale_color_manual(values = fr_cols, name = "Fr") +
  labs(x = "Re", y = "Average log(target) over St")
```



Parallel lines would mean no interaction. For variance, skewness and kurtosis the lines are far from parallel: strong gravity (Fr = 0.052) has a large effect at Re = 90, a smaller one at Re = 224 and almost none at Re = 398. So **Re and Fr interact**. In the St plots above, the strong-gravity curves for the mean also have a different shape (they flatten at large St), a possible **St** × Fr interaction.

Fr = 0.3 behaves like no gravity

In every panel, the Fr = 0.3 and Fr = Inf curves lie on top of each other; only strong gravity (Fr = 0.052) stands apart. A simpler, physically meaningful coding is a two-level gravity variable:

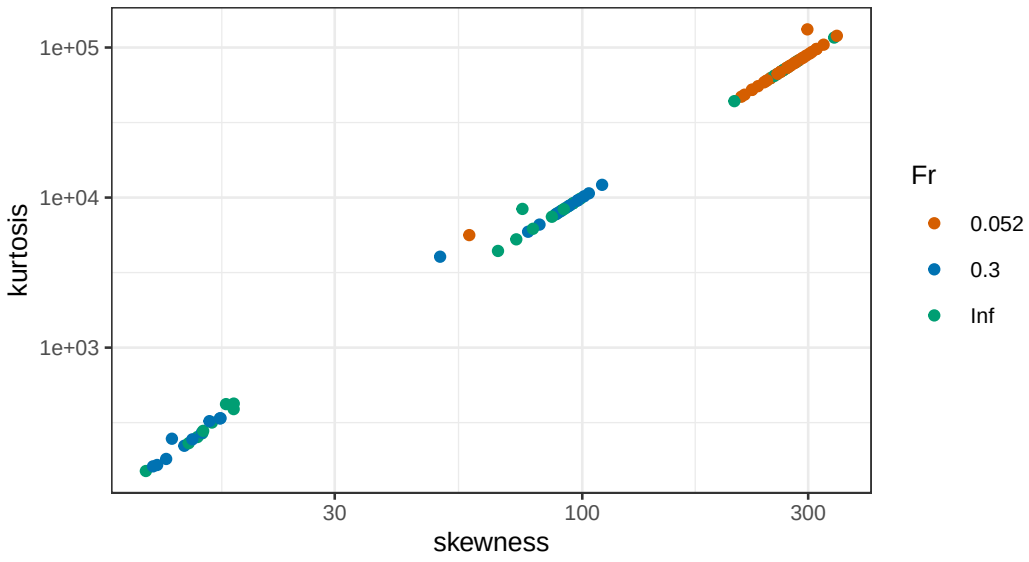
```
add_grav <- function(d) {
  d$Grav <- factor(ifelse(d$Fr == 0.052, "strong", "weak/none"),
                   levels = c("weak/none", "strong"))
  d
}
train <- add_grav(train)
test  <- add_grav(test)
table(Re = train$Re_f, Gravity = train$Grav)
```

```
     Gravity
Re    weak/none strong
  90         19     12
  224        25     14
  398         9     10
```

This also removes the empty Re = 398, Fr = 0.3 cell, so every Re × gravity combination has data. (Section 4 checks that this coding predicts as well as the three-level Fr.)

Skewness and kurtosis carry the same information

```
ggplot(train, aes(skewness, kurtosis, color = Fr_f)) +
  geom_point() + scale_x_log10() + scale_y_log10() +
  scale_color_manual(values = fr_cols, name = "Fr")
```




```
coef(lm(log_kurtosis ~ log_skewness, data = train))
```

```
 (Intercept) log_skewness
    0.239727     1.958474
```

```
cor(train$log_kurtosis, train$log_skewness)
```

```
[1] 0.9990431
```

log(kurtosis) $\approx$ 2 log(skewness), so kurtosis $\approx$ skewness^2: expect the two models to tell the same story.

EDA summary

- **Transformation:** log of every target; log(St) as the St variable.
- **Nonlinearity:** strong in St, even on the log scale $\Rightarrow$ polynomial terms in log(St).
- **Interactions:** Re $\times$ gravity (clear); St $\times$ gravity (possible, mainly for the mean).
- **Simplification:** Fr $= 0.3 \approx$ no gravity $\Rightarrow$ two-level gravity variable.

Candidate models

All models use log(target) as the response. Each step adds one idea, from simplest to most complex. The same models are fit for each of the four targets.

Model	Formula (right-hand side)	Idea added
M1 Linear	St + Re + Grav	Baseline: additive and linear
M2 Log St	log(St) + Re + Grav	Log-transformed St
M3 Polynomial	poly(log St, 3) + Re + Grav	Curved St effect
M4 Re × Grav	poly(log St, 3) + Re × Grav	Gravity effect depends on Re
M5 + St × Grav	poly(log St, 3) × Grav + Re × Grav	St curve depends on gravity
M6 Full interaction	poly(log St, 3) × Re × Grav	Separate curve per setting
M7 Lasso	M6 terms, lasso penalty	Shrink M6, some terms to 0
M8 Ridge	M6 terms, ridge penalty	Shrink all M6 terms

```
rhs <- c(
  M1 = "St + Re_f + Grav",
  M2 = "log_St + Re_f + Grav",
  M3 = "poly(log_St, 3) + Re_f + Grav",
  M4 = "poly(log_St, 3) + Re_f * Grav",
  M5 = "poly(log_St, 3) * Grav + Re_f * Grav",
  M6 = "poly(log_St, 3) * Re_f * Grav"
)
model_names <- c(M1 = "M1 Linear", M2 = "M2 Log St", M3 = "M3 Polynomial",
                 M4 = "M4 Re x Grav", M5 = "M5 + St x Grav",
                 M6 = "M6 Full interaction", M7 = "M7 Lasso",
                 M8 = "M8 Ridge")
make_formula <- function(y, m) as.formula(paste(y, "~", rhs[[m]]))

# Design matrix for lasso and ridge (the M6 terms, raw powers of log St)
x_matrix <- function(d) {
  model.matrix(~ poly(log_St, 3, raw = TRUE) * Re_f * Grav, data = d)[, -1]
}
```

Fit on the full training data

```
train_fit <- expand.grid(model = names(rhs), target = names(responses),
                         stringsAsFactors = FALSE)
```

```r
train_fit$n_coef <- NA; train_fit$train_RMSE <- NA; train_fit$adj_R2 <- NA
for (i in seq_len(nrow(train_fit))) {
  fit <- lm(make_formula(responses[[train_fit$target[i]]],
                         train_fit$model[i]), data = train)
  train_fit$n_coef[i]     <- length(coef(fit))
  train_fit$train_RMSE[i] <- sqrt(mean(resid(fit)^2))
  train_fit$adj_R2[i]     <- summary(fit)$adj.r.squared
}
train_fit |>
  mutate(train_RMSE = round(train_RMSE, 3)) |>
  select(model, n_coef, target, train_RMSE) |>
  pivot_wider(names_from = target, values_from = train_RMSE) |>
  mutate(model = model_names[model]) |>
  knitr::kable(caption = "Training RMSE (log scale)")
```

Table 2: Training RMSE (log scale)

model	n_coef	Mean	Variance	Skewness	Kurtosis
M1 Linear	5	0.157	1.766	0.559	1.082
M2 Log St	5	0.134	1.611	0.566	1.093
M3 Polynomial	7	0.126	1.486	0.555	1.080
M4 Re x Grav	9	0.086	0.595	0.174	0.315
M5 + St x Grav	12	0.061	0.526	0.159	0.288
M6 Full interaction	24	0.046	0.266	0.075	0.131

```r
train_fit |>
  mutate(adj_R2 = round(adj_R2, 3)) |>
  select(model, target, adj_R2) |>
  pivot_wider(names_from = target, values_from = adj_R2) |>
  mutate(model = model_names[model]) |>
  knitr::kable(caption = "Adjusted R-squared")
```

Table 3: Adjusted R-squared

model	Mean	Variance	Skewness	Kurtosis
M1 Linear	0.995	0.760	0.741	0.748
M2 Log St	0.996	0.800	0.735	0.742
M3 Polynomial	0.997	0.826	0.738	0.742
M4 Re x Grav	0.998	0.971	0.974	0.978

model	Mean	Variance	Skewness	Kurtosis
M5 + St x Grav	0.999	0.977	0.977	0.981
M6 Full interaction	0.999	0.993	0.994	0.995

Training error always falls as models get more complex, so it cannot be used to choose a model. Cross-validation estimates error on new data.

Cross-validation and model selection

10-fold cross-validation

Each run is held out once; the model is fit on the other nine folds and predicts the held-out runs. For lasso and ridge, the penalty λ is chosen by an inner cross-validation inside each training fold, so their CV error is honest too.

```
set.seed(325)
folds <- sample(rep(1:10, length.out = nrow(train)))

cv_one <- function(y, m) {
  pred <- numeric(nrow(train))
  for (k in 1:10) {
    tr <- train[folds != k, ]; te <- train[folds == k, ]
    if (m %in% names(rhs)) {
      fit <- lm(make_formula(y, m), data = tr)
      pred[folds == k] <- predict(fit, newdata = te)
    } else {
      set.seed(k)
      fit <- cv.glmnet(x_matrix(tr), tr[[y]], alpha = ifelse(m == "M7", 1, 0))
      pred[folds == k] <- predict(fit, x_matrix(te), s = "lambda.min")
    }
  }
  err <- train[[y]] - pred
  fold_rmse <- tapply(err, folds, function(e) sqrt(mean(e^2)))
  c(cv_RMSE = sqrt(mean(err^2)), SE = sd(fold_rmse) / sqrt(10))
}

cv_results <- expand.grid(model = names(model_names),
                          target = names(responses),
                          stringsAsFactors = FALSE)
cv_out <- t(mapply(function(m, t) cv_one(responses[[t]], m),
```

```
                    cv_results$model, cv_results$target))
cv_results <- cbind(cv_results, cv_out)

cv_results |>
  mutate(cv_RMSE = round(cv_RMSE, 3)) |>
  select(model, target, cv_RMSE) |>
  pivot_wider(names_from = target, values_from = cv_RMSE) |>
  mutate(model = model_names[model]) |>
  knitr::kable(caption = "10-fold CV RMSE (log scale; lower is better)")
```

Table 4: 10-fold CV RMSE (log scale; lower is better)

model	Mean	Variance	Skewness	Kurtosis
M1 Linear	0.168	1.902	0.608	1.176
M2 Log St	0.139	1.735	0.614	1.187
M3 Polynomial	0.139	1.688	0.619	1.200
M4 Re x Grav	0.101	0.791	0.229	0.410
M5 + St x Grav	0.074	0.788	0.277	0.511
M6 Full interaction	0.148	0.684	0.231	0.405
M7 Lasso	0.091	0.939	0.239	0.411
M8 Ridge	0.377	0.971	0.465	0.893

An RMSE of r on the log scale means predictions are typically off by a factor of about e^r (for example, $0.10 \approx 10\%$).

Adding the Re $\times$ gravity interaction (M4) cuts the CV error for variance, skewness and kurtosis by more than half; the polynomial and log terms alone (M2, M3) barely help. Ridge (M8) does poorly because it shrinks **every** coefficient toward zero, including the large and real Re and gravity effects, which adds bias. Lasso (M7) can keep those large terms and drop small ones, so it does much better than ridge, but not better than the simpler M4 or M5.

Check: two-level gravity vs three-level Fr

```
cv_formula <- function(f) {
  pred <- numeric(nrow(train))
  for (k in 1:10) {
    fit <- lm(f, data = train[folds != k, ])
    pred[folds == k] <- predict(fit, newdata = train[folds == k, ])
  }
```

```
  sqrt(mean((train[[all.vars(f)[1]]] - pred)^2))
}
sapply(responses, function(y) c(
  "M4 with 3-level Fr" = cv_formula(as.formula(
    paste(y, "~ poly(log_St, 3) + Re_f * Fr_f"))),
  "M4 with gravity"    = cv_formula(as.formula(
    paste(y, "~ poly(log_St, 3) + Re_f * Grav"))))) |> round(3)
```

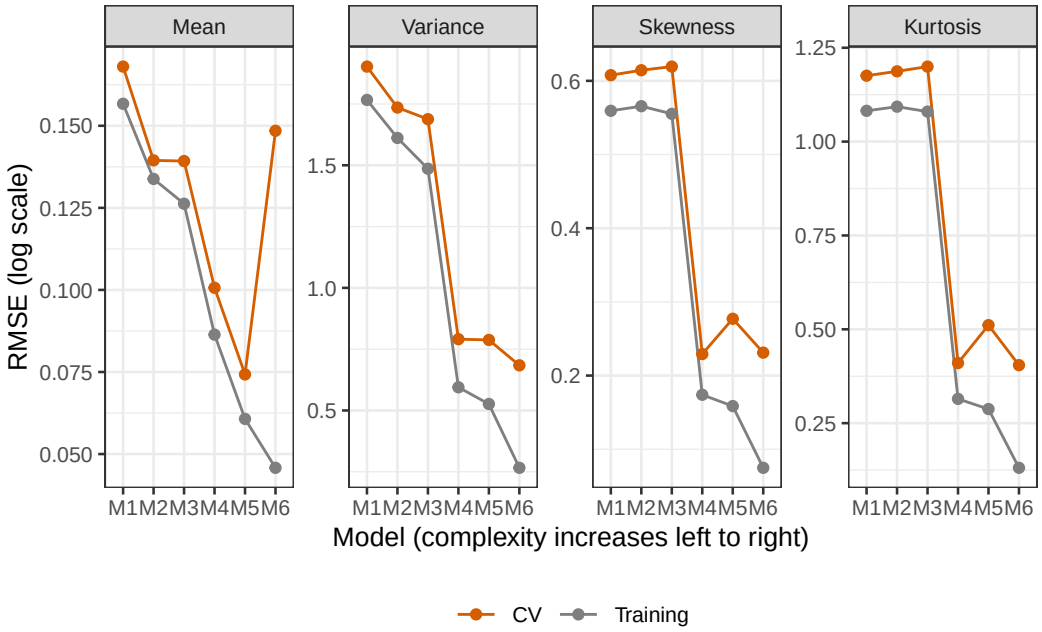
```
                    Mean Variance Skewness Kurtosis
M4 with 3-level Fr 0.098    0.798    0.229     0.41
M4 with gravity    0.101    0.791    0.229     0.41
```

The two codings give almost the same CV error, so the simpler gravity variable loses nothing.

Bias-variance trade-off

```
bv <- cv_results |>
  filter(model %in% names(rhs)) |>
  left_join(train_fit, by = c("model", "target")) |>
  select(model, target, n_coef, Training = train_RMSE, CV = cv_RMSE) |>
  pivot_longer(c(Training, CV), names_to = "error", values_to = "rmse") |>
  mutate(target = factor(target, levels = names(responses)))

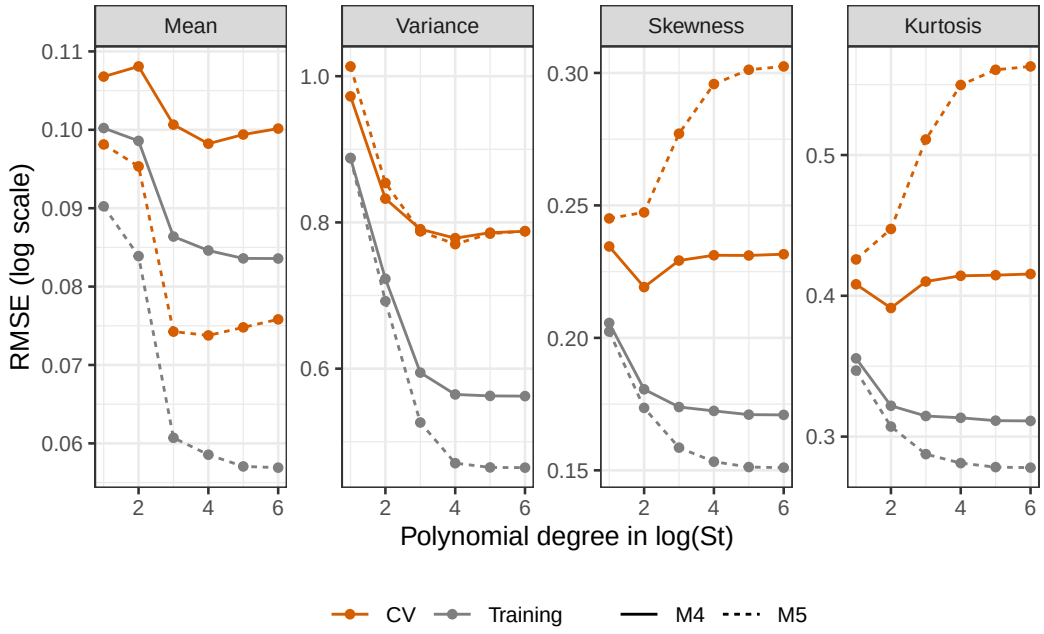
ggplot(bv, aes(model, rmse, color = error, group = error)) +
  geom_line() + geom_point() +
  facet_wrap(~ target, scales = "free_y", nrow = 1) +
  scale_color_manual(values = c(Training = "grey50", CV = "#D55E00")) +
  labs(x = "Model (complexity increases left to right)",
       y = "RMSE (log scale)", color = NULL) +
  theme(legend.position = "bottom")
```



Training error keeps falling as complexity grows. CV error falls sharply at M4 (adding Re × gravity removes a large **bias**) and then flattens or rises: the full-interaction model M6 fits each setting separately with only about 10 runs per setting, so its **variance** is high (for the mean its CV error is worse than M4's).

```
# Within the M4 and M5 structures: how many polynomial terms in log St?
deg_grid <- expand.grid(degree = 1:6, structure = c("M4", "M5"),
                        target = names(responses), stringsAsFactors = FALSE)
deg_rhs <- c(M4 = "poly(log_St, %d) + Re_f * Grav",
             M5 = "poly(log_St, %d) * Grav + Re_f * Grav")
deg_res <- t(mapply(function(d, s, t) {
  f <- as.formula(paste(responses[[t]], "~", sprintf(deg_rhs[[s]], d)))
  pred <- numeric(nrow(train))
  for (k in 1:10) {
    fit <- lm(f, data = train[folds != k, ])
    pred[folds == k] <- predict(fit, newdata = train[folds == k, ])
  }
  err <- train[[responses[[t]]]] - pred
  c(Training = sqrt(mean(resid(lm(f, data = train))^2)),
    CV = sqrt(mean(err^2)),
    SE = sd(tapply(err, folds, function(e) sqrt(mean(e^2)))) / sqrt(10))
}, deg_grid$degree, deg_grid$structure, deg_grid$target))
deg_grid <- cbind(deg_grid, deg_res)
```

```
deg_grid |>
  pivot_longer(c(Training, CV), names_to = "error", values_to = "rmse") |>
  mutate(target = factor(target, levels = names(responses))) |>
  ggplot(aes(degree, rmse, color = error, linetype = structure)) +
  geom_line() + geom_point(size = 1.2) +
  facet_wrap(~ target, scales = "free_y", nrow = 1) +
  scale_color_manual(values = c(Training = "grey50", CV = "#D55E00")) +
  labs(x = "Polynomial degree in log(St)", y = "RMSE (log scale)",
       color = NULL, linetype = NULL) +
  theme(legend.position = "bottom")
```



The same pattern appears for the polynomial degree: training error keeps falling, while CV error stops improving after a degree of 2 to 4.

Model selection: the one-standard-error rule

For each target, find the model with the lowest CV RMSE, then choose the **simplest** model whose CV RMSE is within one standard error of it. This favors interpretability when the data cannot tell models apart.

```
# Order of complexity; lasso and ridge use the M6 terms, so they rank with M6
complexity <- c(M1 = 1, M2 = 2, M3 = 3, M4 = 4, M5 = 5,
                M6 = 6, M7 = 6, M8 = 6)
selected <- cv_results |>
  group_by(target) |>
  mutate(threshold = cv_RMSE[which.min(cv_RMSE)] + SE[which.min(cv_RMSE)]) |>
  filter(cv_RMSE <= threshold) |>
  slice_min(complexity[model], n = 1, with_ties = FALSE) |>
  ungroup()
selected |>
  transmute(target, model = model_names[model], cv_RMSE = round(cv_RMSE, 3),
            threshold = round(threshold, 3)) |>
  knitr::kable(caption = "Selected model per target (one-SE rule)")
```

Table 5: Selected model per target (one-SE rule)

target	model	cv_RMSE	threshold
Kurtosis	M4 Re x Grav	0.410	0.498
Mean	M5 + St x Grav	0.074	0.081
Skewness	M4 Re x Grav	0.229	0.278
Variance	M4 Re x Grav	0.791	0.804

Then choose the polynomial degree for each selected model. The degree is an ordinary tuning parameter, so we take the degree with the **lowest** CV error. (The one-SE rule would pick degree 1 for skewness and kurtosis, a straight line in log St, which the EDA plots show is too simple.)

```
chosen <- deg_grid |>
  inner_join(select(selected, target, structure = model), by =
               c("target", "structure")) |>
  group_by(target) |>
  slice_min(CV, n = 1, with_ties = FALSE) |>
  ungroup()
chosen |>
  transmute(target, structure, degree, cv_RMSE = round(CV, 3)) |>
  knitr::kable(caption = "Final models")
```

Table 6: Final models

target	structure	degree	cv_RMSE
Kurtosis	M4	2	0.391
Mean	M5	4	0.074
Skewness	M4	2	0.219
Variance	M4	4	0.778

```
final_formulas <- setNames(lapply(names(responses), function(t) {
  r <- chosen[chosen$target == t, ]
  as.formula(paste(responses[[t]], "~",
                   sprintf(deg_rhs[[r$structure]], r$degree)))
}), names(responses))
final_formulas
```

```
$Mean
log_mean ~ poly(log_St, 4) * Grav + Re_f * Grav
<environment: 0x11e166db8>

$Variance
log_variance ~ poly(log_St, 4) + Re_f * Grav
<environment: 0x11e16dfb8>

$Skewness
log_skewness ~ poly(log_St, 2) + Re_f * Grav
<environment: 0x11e350460>

$Kurtosis
log_kurtosis ~ poly(log_St, 2) + Re_f * Grav
<environment: 0x11e35b7d8>
```

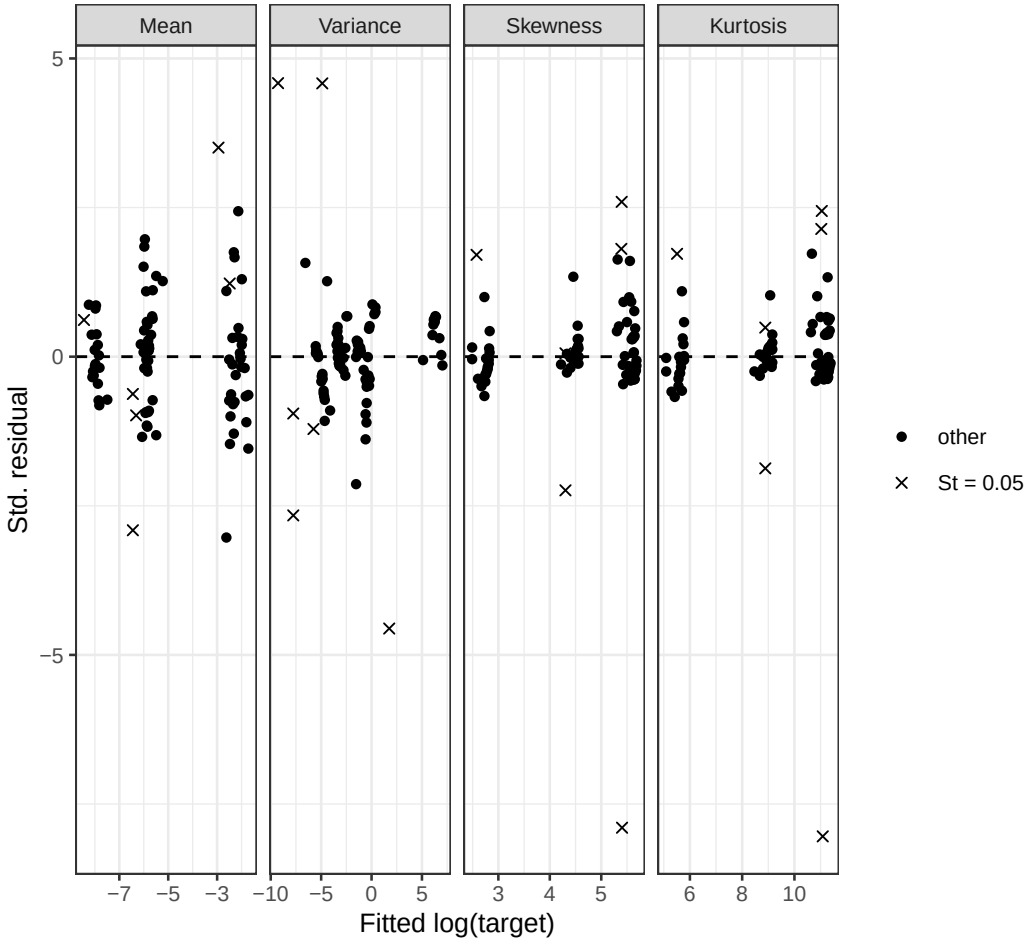
Diagnostics of the final models

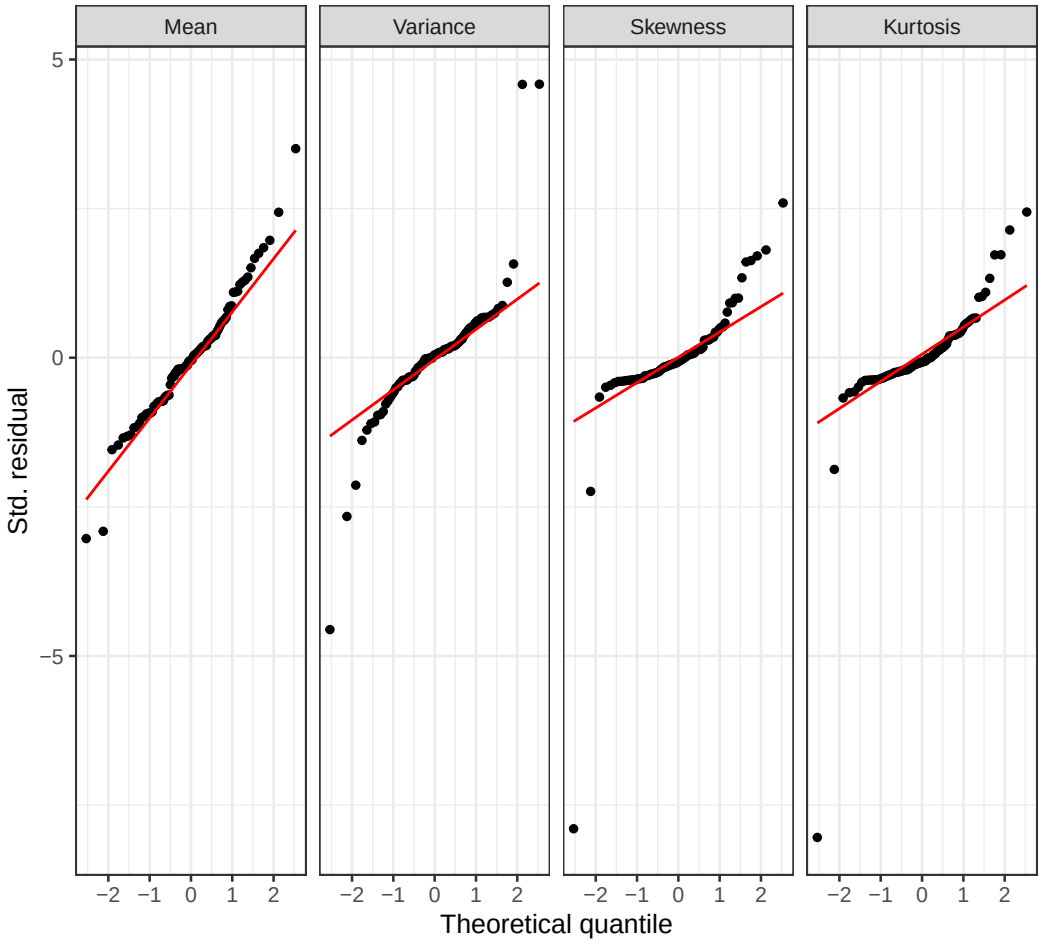
```
final_fits <- lapply(final_formulas, lm, data = train)

diag_df <- bind_rows(lapply(names(final_fits), function(t) {
  data.frame(target = t, fitted = fitted(final_fits[[t]]),
             resid = rstandard(final_fits[[t]]), St = train$St)
})) |> mutate(target = factor(target, levels = names(responses)),
```

```
            edge = ifelse(St == 0.05, "St = 0.05", "other"))

p1 <- ggplot(diag_df, aes(fitted, resid, shape = edge)) +
  geom_hline(yintercept = 0, linetype = "dashed") + geom_point() +
  facet_wrap(~ target, scales = "free_x", nrow = 1) +
  scale_shape_manual(values = c("St = 0.05" = 4, other = 16)) +
  labs(x = "Fitted log(target)", y = "Std. residual", shape = NULL)
p2 <- ggplot(diag_df, aes(sample = resid)) +
  stat_qq(size = 1) + stat_qq_line(color = "red") +
  facet_wrap(~ target, nrow = 1) +
  labs(x = "Theoretical quantile", y = "Std. residual")
p1; p2
```





The residuals show no remaining pattern by fitted value. The few large residuals are runs at $\mathrm{St} = 0.05$, the smallest inertia, where clustering is just beginning and the cluster summaries are noisy; they make the residual tails heavier than normal.

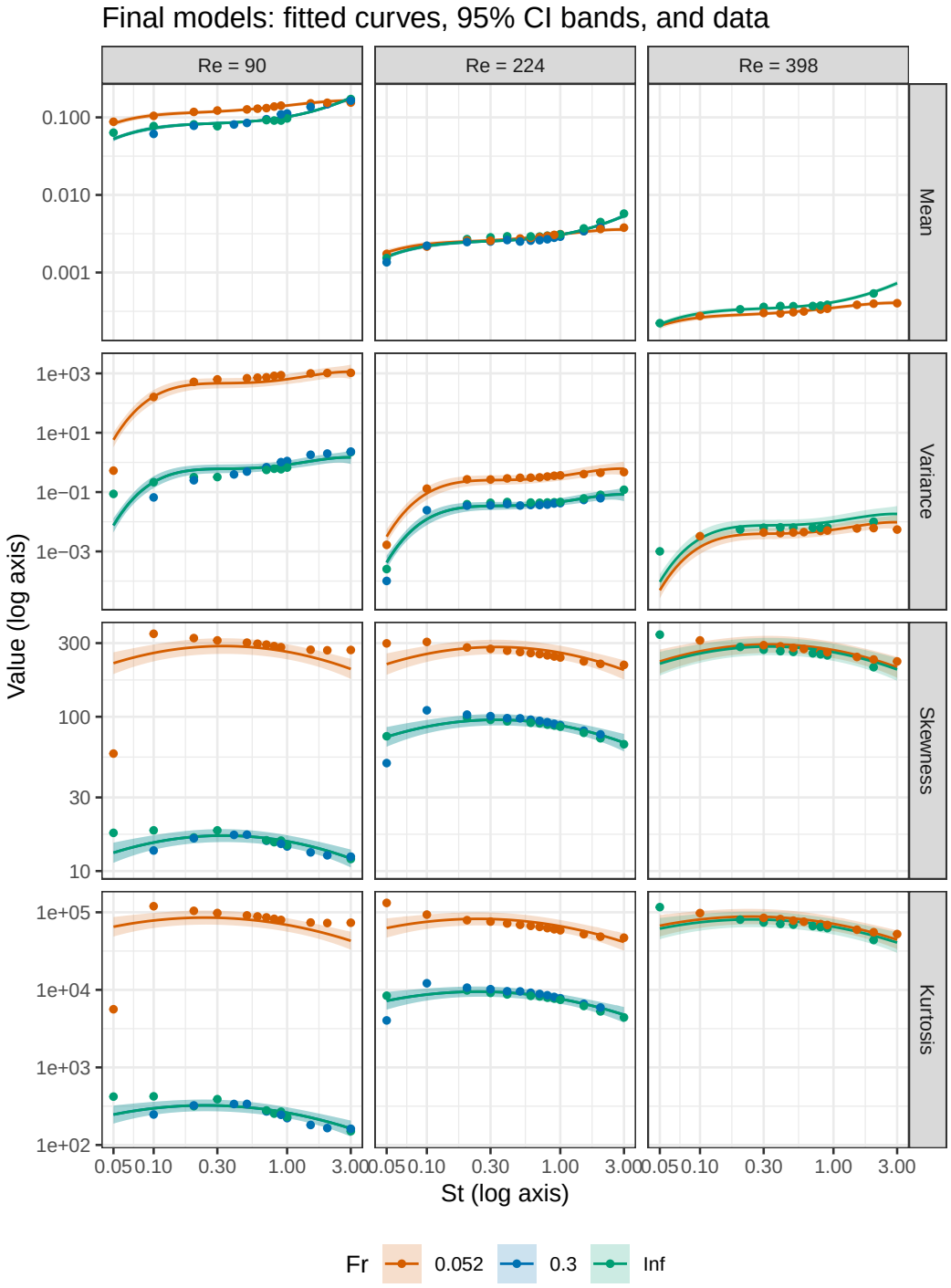
Scientific inference

Fitted effects

```
settings <- unique(train[, c("Re", "Fr", "Re_f", "Fr_f", "Grav")])
grid <- merge(settings, data.frame(St = exp(seq(log(0.05), log(3),
                                                length.out = 60))))
grid$log_St <- log(grid$St)
```

```r
curves <- bind_rows(lapply(names(final_fits), function(t) {
  p <- predict(final_fits[[t]], newdata = grid, interval = "confidence")
  cbind(grid, target = t, fit = exp(p[, "fit"]),
        lwr = exp(p[, "lwr"]), upr = exp(p[, "upr"]))
}))
obs <- long |> mutate(target = recode(target, mean = "Mean",
  variance = "Variance", skewness = "Skewness", kurtosis = "Kurtosis"))
lv <- names(responses)
curves$target <- factor(curves$target, levels = lv)
obs$target <- factor(obs$target, levels = lv)

ggplot(curves, aes(St, fit, color = Fr_f, fill = Fr_f)) +
  geom_ribbon(aes(ymin = lwr, ymax = upr), alpha = 0.2, color = NA) +
  geom_line() +
  geom_point(data = obs, aes(St, value, color = Fr_f), inherit.aes = FALSE,
             size = 1) +
  facet_grid(target ~ Re_f, scales = "free_y",
             labeller = labeller(Re_f = function(x) paste("Re =", x))) +
  scale_x_log10(breaks = c(0.05, 0.1, 0.3, 1, 3)) + scale_y_log10() +
  scale_color_manual(values = fr_cols, name = "Fr") +
  scale_fill_manual(values = fr_cols, name = "Fr") +
  labs(x = "St (log axis)", y = "Value (log axis)",
       title = "Final models: fitted curves, 95% CI bands, and data") +
  theme(legend.position = "bottom")
```


Final models: fitted curves, 95% CI bands, and data
Re = 90
Re = 224
Re = 398
Mean
Variance
Skewness
Kurtosis
Value (log axis)
St (log axis)
0.100
0.010
0.001
1e+03
1e+01
1e−01
1e−03
300
100
30
10
1e+05
1e+04
1e+03
1e+02
0.05
0.10
0.30
1.00
3.00
Fr
0.052
0.3
Inf

Effect sizes

Each effect compares two settings, holding the others fixed, as a **fold-change**: the ratio of predicted values (1 = no effect).

```
setting <- function(St, Re, Fr) {
  d <- data.frame(St = St, Re = Re, Fr = Fr)
  add_grav(encode(d))
}
fold_change <- function(fit, a, b) {
  # log-scale difference between settings a and b, with a 95% CI
  X <- model.matrix(delete.response(terms(fit)), rbind(a, b),
                    xlev = fit$xlevels)
  cvec <- X[1, ] - X[2, ]
  est <- sum(cvec * coef(fit))
  se <- sqrt(drop(t(cvec) %*% vcov(fit) %*% cvec))
  q <- qt(0.975, df.residual(fit))
  sprintf("%.3g (%.2g, %.2g)", exp(est), exp(est - q * se), exp(est + q * se))
}
contrasts <- list(
  "Re 90 -> 224 (no gravity)" = list(setting(0.5, 224, Inf), setting(0.5, 90, Inf)),
  "Re 224 -> 398 (no gravity)" = list(setting(0.5, 398, Inf), setting(0.5, 224, Inf)),
  "Strong vs no gravity, Re 90" = list(setting(0.5, 90, 0.052), setting(0.5, 90, Inf)),
  "Strong vs no gravity, Re 224" = list(setting(0.5, 224, 0.052), setting(0.5, 224, Inf)),
  "Strong vs no gravity, Re 398" = list(setting(0.5, 398, 0.052), setting(0.5, 398, Inf)),
  "St 0.1 -> 1 (no gravity)" = list(setting(1, 224, Inf), setting(0.1, 224, Inf)),
  "St 1 -> 3 (no gravity)" = list(setting(3, 224, Inf), setting(1, 224, Inf))
)
effects <- sapply(final_fits, function(fit)
  sapply(contrasts, function(cc) fold_change(fit, cc[[1]], cc[[2]])))
knitr::kable(effects, caption = paste("Fold-change (95% CI); St = 0.5",
                                      "unless St is the input changed"))
```

Table 7: Fold-change (95% CI); St = 0.5 unless St is the input changed

	Mean	Variance	Skewness	Kurtosis
Re 90 -> 224 (no gravity)	0.0302 (0.029, 0.031)	0.0563 (0.039, 0.081)	5.63 (5, 6.3)	29.3 (24, 36)
Re 224 -> 398 (no gravity)	0.135 (0.13, 0.14)	0.221 (0.14, 0.35)	2.98 (2.6, 3.5)	8.53 (6.6, 11)
Strong vs no gravity, Re 90	1.43 (1.3, 1.5)	765 (4.9e+02, 1.2e+03)	16.9 (15, 19)	264 (2.1e+02, 3.4e+02)

	Mean	Variance	Skewness	Kurtosis
Strong vs no gravity, Re 224	1.04 (0.98, 1.1)	7.42 (5, 11)	2.96 (2.6, 3.4)	8.68 (6.9, 11)
Strong vs no gravity, Re 398	0.866 (0.81, 0.93)	0.525 (0.3, 0.91)	1.03 (0.87, 1.2)	1.09 (0.8, 1.5)
St 0.1 -> 1 (no gravity)	1.4 (1.3, 1.5)	3.84 (2.4, 6)	1.02 (0.93, 1.1)	0.877 (0.75, 1)
St 1 -> 3 (no gravity)	1.76 (1.6, 1.9)	1.78 (1, 3.1)	0.774 (0.7, 0.86)	0.625 (0.52, 0.75)

Findings

How does each parameter affect the cluster-volume distribution?

- **Re (turbulence intensity)** has the largest effect. Higher Re makes the mean (normalized) cluster volume much smaller but makes the distribution far more skewed and heavy-tailed: stronger turbulence produces many smaller clusters plus rare very large ones. The effect is nonlinear in Re (the first step changes the targets much more than the second).
- **Fr (gravity)** matters only when gravity is strong (Fr = 0.052); weak gravity behaves like no gravity. The strong-gravity effect depends on Re: it is very large at Re = 90 (variance about 750 times larger), smaller at Re = 224, and gone by Re = 398, because intense turbulence breaks up the structures that settling creates.
- **St (particle inertia)** has a clearly nonlinear effect. The mean and variance increase with St, fastest at small St; skewness and kurtosis stay roughly flat at small St and decline at large St (skewness falls by about a quarter from St = 1 to 3).

Are the effects the same for all four summaries? No.

- Re moves the mean down but skewness and kurtosis up.
- Gravity barely changes the mean but multiplies the variance and tail measures: it changes how unequal cluster sizes are, not the typical size.
- Only the mean needs a St × gravity interaction: under strong gravity, the mean cluster volume stops growing at large St.
- Kurtosis follows skewness closely (kurtosis $\approx$ skewness^2).

Final predictions on the test set

The final models are refit on all 89 training runs and used to predict the 23 test settings. The case study asks for the four summary statistics, so the raw moments $E[X^k]$ are not needed

for the test set. Predictions are back-transformed with `exp()`, so they estimate the median of each summary at that setting.

```
pred_log <- sapply(final_fits, function(fit)
  predict(fit, newdata = test, interval = "prediction"), simplify = FALSE)

predictions <- data.frame(
  St = test$St, Re = test$Re, Fr = test$Fr,
  mean     = exp(pred_log$Mean[, "fit"]),
  sd       = sqrt(exp(pred_log$Variance[, "fit"])),
  variance = exp(pred_log$Variance[, "fit"]),
  skewness = exp(pred_log$Skewness[, "fit"]),
  kurtosis = exp(pred_log$Kurtosis[, "fit"])
)

# Saved next to this file (not in data/), so the original
# data/data-test.csv is not overwritten
write.csv(predictions, "data-test.csv", row.names = FALSE)

predictions |>
  mutate(across(mean:kurtosis, \(x) format(signif(x, 3), trim = TRUE))) |>
  knitr::kable(caption = "Test-set predictions", align = "r")
```

Table 8: Test-set predictions

St	Re	Fr	mean	sd	variance	skewness	kurtosis
0.05	398	0.052	0.000206	0.00701	4.92e-05	228.0	67100
0.20	398	0.052	0.000285	0.06000	3.60e-03	289.0	87800
0.70	398	0.052	0.000329	0.06710	4.50e-03	283.0	78000
1.00	398	0.052	0.000351	0.07350	5.40e-03	270.0	70900
0.10	398	Inf	0.000297	0.05180	2.68e-03	257.0	74200
0.60	398	Inf	0.000369	0.09030	8.15e-03	278.0	74000
1.00	398	Inf	0.000415	0.10100	1.03e-02	262.0	65100
1.50	398	Inf	0.000483	0.11700	1.36e-02	243.0	56400
3.00	398	Inf	0.000728	0.13500	1.83e-02	203.0	40700
3.00	224	0.300	0.005380	0.28800	8.30e-02	67.9	4770
0.10	224	Inf	0.002190	0.11000	1.22e-02	86.2	8700
0.50	224	Inf	0.002660	0.18800	3.55e-02	94.5	8960
0.40	90	0.052	0.123000	21.70000	4.71e+02	287.0	83200
1.00	90	0.052	0.142000	25.20000	6.34e+02	264.0	68700
0.05	90	0.300	0.052600	0.08690	7.55e-03	13.1	246

St	Re	Fr	mean	sd	variance	skewness	kurtosis
0.30	90	0.300	0.084100	0.77800	6.05e-01	17.0	322
0.60	90	0.300	0.090400	0.81000	6.57e-01	16.6	296
0.80	90	0.300	0.095600	0.85600	7.32e-01	16.1	277
0.40	90	Inf	0.086000	0.78400	6.15e-01	16.9	315
0.50	90	Inf	0.088100	0.79400	6.31e-01	16.8	306
0.60	90	Inf	0.090400	0.81000	6.57e-01	16.6	296
1.50	90	Inf	0.118000	1.05000	1.09e+00	14.5	226
2.00	90	Inf	0.137000	1.15000	1.32e+00	13.5	200

```
# Sanity checks: all predictions positive and finite
all(is.finite(as.matrix(predictions[, 4:8]))) &&
  all(predictions[, 4:8] > 0)
```

```
[1] TRUE
```

Uncertainty in the predictions

The case study asks for uncertainty in the predictive results. The table below gives a 95% prediction interval for each summary (shown for the mean and variance; the same is done for all four). Four test settings use an St outside the range observed for their (Re, Fr) setting, so they are extrapolations.

```
st_range <- train |> group_by(Re, Fr) |>
  summarise(st_min = min(St), st_max = max(St), .groups = "drop")
extrapolation <- test |> left_join(st_range, by = c("Re", "Fr")) |>
  mutate(ext = St < st_min | St > st_max) |> pull(ext)

data.frame(
  St = test$St, Re = test$Re, Fr = test$Fr,
  mean = sprintf("%.3g (%.3g, %.3g)", exp(pred_log$Mean[, "fit"]),
                 exp(pred_log$Mean[, "lwr"]), exp(pred_log$Mean[, "upr"])),
  variance = sprintf("%.3g (%.3g, %.3g)", exp(pred_log$Variance[, "fit"]),
                     exp(pred_log$Variance[, "lwr"]),
                     exp(pred_log$Variance[, "upr"])),
  extrapolation = extrapolation
) |>
  knitr::kable(caption = "Predictions with 95% prediction intervals")
```

Table 9: Predictions with 95% prediction intervals

St	Re	Fr	mean	variance	extrapolation
0.05	398	0.052	0.000206 (0.000175, 0.000242)	4.92e-05 (1.28e-05, 0.000189)	TRUE
0.20	398	0.052	0.000285 (0.000248, 0.000327)	0.0036 (0.00101, 0.0129)	FALSE
0.70	398	0.052	0.000329 (0.000287, 0.000376)	0.0045 (0.00128, 0.0158)	FALSE
1.00	398	0.052	0.000351 (0.000306, 0.000402)	0.0054 (0.00153, 0.0191)	FALSE
0.10	398	Inf	0.000297 (0.000256, 0.000343)	0.00268 (0.000715, 0.01)	FALSE
0.60	398	Inf	0.000369 (0.000323, 0.000423)	0.00815 (0.00231, 0.0288)	FALSE
1.00	398	Inf	0.000415 (0.000362, 0.000475)	0.0103 (0.00289, 0.0366)	FALSE
1.50	398	Inf	0.000483 (0.00042, 0.000555)	0.0136 (0.00376, 0.0492)	FALSE
3.00	398	Inf	0.000728 (0.000627, 0.000847)	0.0183 (0.00479, 0.07)	TRUE
3.00	224	0.300	0.00538 (0.00465, 0.00622)	0.083 (0.0226, 0.304)	TRUE
0.10	224	Inf	0.00219 (0.0019, 0.00252)	0.0122 (0.00339, 0.0436)	FALSE
0.50	224	Inf	0.00266 (0.00233, 0.00303)	0.0355 (0.0104, 0.121)	FALSE
0.40	90	0.052	0.123 (0.107, 0.141)	471 (134, 1.65e+03)	FALSE
1.00	90	0.052	0.142 (0.124, 0.163)	634 (181, 2.22e+03)	FALSE
0.05	90	0.300	0.0526 (0.0455, 0.0608)	0.00755 (0.00203, 0.028)	TRUE
0.30	90	0.300	0.0841 (0.0736, 0.0961)	0.605 (0.175, 2.09)	FALSE
0.60	90	0.300	0.0904 (0.0792, 0.103)	0.657 (0.191, 2.26)	FALSE
0.80	90	0.300	0.0956 (0.0838, 0.109)	0.732 (0.213, 2.51)	FALSE
0.40	90	Inf	0.086 (0.0753, 0.0983)	0.615 (0.178, 2.12)	FALSE
0.50	90	Inf	0.0881 (0.0771, 0.101)	0.631 (0.183, 2.17)	FALSE
0.60	90	Inf	0.0904 (0.0792, 0.103)	0.657 (0.191, 2.26)	FALSE
1.50	90	Inf	0.118 (0.103, 0.135)	1.09 (0.315, 3.8)	FALSE
2.00	90	Inf	0.137 (0.12, 0.156)	1.32 (0.38, 4.59)	FALSE

The prediction intervals come from the linear models and assume the typical residual size applies everywhere. Cross-validation showed errors are larger at the edges of each setting's St range, so the intervals for the four extrapolated settings should be read as optimistic.